Run guide: the 132-run program on four more backbones

What to run, in which order, when, and how to check it

Ankit Kumar, University of Agder • 1 October 2026 • for the package `kernelroute-models.zip`, Windows PowerShell, one RTX 4070 Ti

The short version

1. **Day 1, about 1 hour:** unzip, install, get the data, commit the pre-registration, run the tests (Section 2).
2. **Day 1, evening:** start the program with one command and leave it running (Section 3):

```
python run_all_models.py
```

3. **Every day, 2 minutes:** check progress and the log (Section 5).
4. **At four check points:** look at each backbone's gate result and build its report as soon as it finishes (Section 4).
5. **At the end:** build the cross-model report and send me the results folder (Section 6).

The program trains 528 networks: 132 per backbone, in the order MobileNetV2, VGG-16, ResNet-50, ConvNeXt-Tiny. Everything is automatic: memory, resuming after a crash, skipping a backbone that does not train. Expect roughly 7 to 10 weeks on one GPU (Section 4).

1 What gets run

Each backbone repeats the ResNet-18 study exactly: the same ten groups, the same arms, six seeds (0 to 5), the same recipe (SGD Nesterov 0.9, lr 0.1, weight decay 5×10^{-4}, batch 128, 5 warm-up epochs then cosine, 150 epochs, crop and flip, bf16).

group	dataset	width	K	temperature	arms	runs
1	CIFAR-100	64	4	1	static, uniform, gap, shuffled, frozen_random	30
2	CIFAR-100	32	4	1	static, uniform, gap	18
3	CIFAR-100	16	4	1	static, uniform, gap	18
4	CIFAR-100	32	2	1	gap	6
5	CIFAR-100	32	8	1	gap	6
6	Tiny-ImageNet	32	4	1	static, uniform, gap	18
7	CIFAR-100	64	4	30→1	gap	6
8	CIFAR-100	32	4	30→1	gap	6
9	CIFAR-100	16	4	30→1	gap	6
10	Tiny-ImageNet	64	4	1	static, uniform, gap (seed by seed)	18
per backbone						132

Inside a group the runner goes arm by arm (all six seeds of `static`, then `uniform`, and so on). Group 10 goes seed by seed, so an interruption still leaves complete static/uniform/gap triplets. The very first run of every backbone is **the gate**: `static`, CIFAR-100, width 64, seed 0.

2 One-time setup (Day 1, about 1 hour)

2.1 Unzip and install

Unzip `kernelroute-models.zip` into a folder, for example `C:\research\kernelroute`. If you still have the ResNet-18 folder, keep it separate; you will point the reports at its `results` folder later. Open PowerShell in the new folder:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu124
pip install -r requirements.txt
python -c "import torch; print(torch.__version__, torch.cuda.is_available(), torch.
    cuda.get_device_name(0))"
```

The last line must print `True` and your GPU's name. If activation is blocked, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once and try again.

For the reports you also need LaTeX: install MiKTeX and make sure `pdflatex` works in a new PowerShell window (`pdflatex -version`).

2.2 Data

CIFAR-100 downloads itself on the first run. Tiny-ImageNet is needed from group 6 onward. If you already have it from the ResNet-18 study, copy the folder `tiny-imagenet-200` into `.\data`. Otherwise:

```
python download_tinyimagenet.py --root .\data
```

2.3 Commit the pre-registration (before the first run finishes)

`PREREGISTRATION_MODELS.md` records the predictions, gate floors and rules dated 30 September 2026. It only counts as a pre-registration if it is time-stamped before results exist. Do one of these, unmodified:

- with git, then push:

  ```
  git add PREREGISTRATION_MODELS.md
  git commit -m "pre-registration, five backbones"
  ```

- or upload it to OSF (osf.io) as a registration;
- or, at minimum, email it to yourself and your supervisors.

2.4 Run the tests (about 20 minutes, CPU is fine)

These prove the code does what the paper says before you spend GPU weeks on it.

```
python tests\test_paths.py
python tests\test_models.py all
python tests\test_train.py all
python tests\test_runner.py all
python tests\test_analysis.py all
python tests\test_report.py all
```

Each one must end with `ok` (or `all tests passed`) and no `Traceback`. Two checks print *skipped* or *built only* on your machine because they compare against files that only existed in my workspace; that is expected.

2.5 Look before you leap

```
python run_all_models.py --dry-run
```

This prints all 528 training commands and runs nothing. Skim it: the first command of each backbone should be the static width-64 seed-0 gate run.

3 Starting the program

3.1 Prepare the PC

- Settings, System, Power: *Sleep: never* and *Screen off* as you like. Plug the laptop in if it is one.
- Pause Windows Update for the longest period offered (Settings, Windows Update). A forced reboot is harmless (the program resumes) but loses up to one epoch.
- Close other GPU programs (games, browser video). They take GPU memory and can cause an out-of-memory error.

3.2 Start

```
cd C:\research\kernelroute
.\.venv\Scripts\Activate.ps1
python run_all_models.py
```

That one command runs all four backbones in order into `.\results_models\<backbone>\`. Leave the window open. To run one backbone only: `python run_all_models.py -models mobilenetv2`.

3.3 What happens automatically

Probe (a few minutes before each group)
The runner tries a few training steps and one evaluation pass of the group's heaviest arm at batch 128. If it runs out of memory, it retries at 64×2, 32×4, 16×8 (gradient accumulation: same data order, same optimiser steps and learning-rate schedule). The choice is saved in `plan.json`, written to `DEVIATIONS.md` if it is not 128, and the log prints a measured time per run for the group.

Gate (the first run of each backbone)
If the static width-64 seed-0 run ends below the floor (MobileNetV2 65 %, VGG-16 65 %, ResNet-50 70 %, ConvNeXt-Tiny 60 %), the runner writes `GATE_FAILED.json`, skips that backbone's other 131 runs, and moves on to the next backbone.

Each run
writes one result file, for example `vgg16__cifar100__w32__K4__const_gap_seed3.json`, and a checkpoint every epoch while it is running.

Divergence
A run whose loss becomes NaN leaves a `.DIVERGED` marker and the program continues. It is reported, never retried.

Out of memory mid-run
The group steps down one micro-batch size from that run on, logs it, and redoes the run.

3.4 Stopping and resuming

Press Ctrl+C at any time, or let Windows reboot. To continue, run **exactly the same command** again. Finished runs are skipped; an interrupted run continues from its last finished epoch. Nothing else to do.

4 When: timeline and check points

The hours below are an **estimate**: the ResNet-18 times you logged, scaled by how much arithmetic each backbone does. MobileNetV2 and ConvNeXt-Tiny are built from depthwise convolutions, which run slower than their arithmetic suggests, so treat their numbers as a lower bound. The log line after each probe gives the measured figure within the first hour of every group; trust that over this table.

when	what runs	est. GPU-h	est. days	check point (you)
Day 1	setup, tests, start	–	–	tests pass; pre-registration committed
Day 1 to 2 or 4	MobileNetV2	35+	1.5 to 4.5	gate result after first run (about 1 h)
next	VGG-16 (BN)	121	about 5	gate; MobileNetV2 report
next	ResNet-50	467	about 20	gate; VGG-16 report
next	ConvNeXt-Tiny	518+	22 to 30+	gate; ResNet-50 report
end	reports	–	1	ConvNeXt report, cross-model report
total		about 1,140+	roughly 7 to 10 weeks on one GPU	

At each check point:

1. **About an hour after a backbone starts:** open its `runner.log` and find the gate result (`finished in ... min, final XX.XX`). Above the floor: nothing to do. Below: see Section 8, *gate failed*.
2. **When a backbone finishes** (the log says `all 132 jobs present`): build its report (Section 6) and read the coverage table on page 1. You can do this while the next backbone trains.
3. **Halfway through a long backbone**, if you are curious: the report also builds from partial results; missing runs show as `--` and are listed.

Two GPUs? The program splits cleanly: run two PowerShell windows,

```
$env:CUDA_VISIBLE_DEVICES=0; python run_all_models.py --models mobilenetv2 vgg16
$env:CUDA_VISIBLE_DEVICES=1; python run_all_models.py --models resnet50 convnext_tiny
```

Use the same GPU model on both, or report which backbone ran on which GPU.

5 Daily routine (2 minutes)

In a second PowerShell window, in the same folder:

```
.\.venv\Scripts\Activate.ps1
python run_all_models.py --status
Get-Content .\results_models\resnet50\runner.log -Tail 15
```

`--status` prints, per backbone, runs done, runs diverged, of 132, and whether the gate failed. It reads files only; it is safe while training runs.

What normal looks like in the log: one line `[n/132] <group> <arm> seed s (micro-batch m); k left` when a run starts and one line `finished in X min, final YY.YY` when it ends. Lines starting with `DEVIATION` are expected for large models on Tiny-ImageNet and are reported automatically. To watch live: add `-Wait` to `Get-Content`.

Worth a look: `nvidia-smi` (GPU in use, memory below 12 GB), and that `k left` keeps going down from one day to the next.

6 Building the reports

6.1 One backbone, as soon as it finishes

```
python report\build_model_report.py --model mobilenetv2 --results .\results_models\
    mobilenetv2
```

Output: `report\out\mobilenetv2\report_mobilenetv2.pdf`, with the same 10 figures and 13 tables as the ResNet-18 paper, plus a coverage table (missing runs, divergences, micro-batch changes) and an equal-parameter table. The folder can be uploaded to Overleaf as it is. Replace `mobilenetv2` with `vgg16`, `resnet50`, `convnext_tiny`.

6.2 ResNet-18 from your original result files

Point it at the `results` folder of the ResNet-18 study (the 132 JSON files):

```
python report\build_model_report.py --model resnet18 --results C:\research\
    kernelroute-stage1\results
python report\build_paper.py --results C:\research\kernelroute-stage1\results
```

The second command rebuilds the paper itself, with the corrected parameter counts and the initialisation disclosure, from your files. It should match `KernelRoute_Report_revised.pdf`.

6.3 All five side by side (at the end)

```
python report\cross.py --results resnet18=C:\research\kernelroute-stage1\results
    mobilenetv2=.\results_models\mobilenetv2 vgg16=.\results_models\vgg16 resnet50=.\
    results_models\resnet50 convnext_tiny=.\results_models\convnext_tiny
```

Output: `report\out\cross\cross_models.pdf`. A backbone whose gate failed appears as a row saying so; it is not dropped.

The analysis refuses to run if two files claim the same run, if the recipe differs inside one backbone, or if a run's parameter count does not match `PARAMS_EXPECTED.md` (wrong width, K or arm). The message names the file. One-epoch probe or test files in the folder are ignored and listed, not averaged.

7 Rules that keep the results valid

1. **Do not change the code, recipe or gate floors once the first run has started.** If something must change, stop, write down what and why with the date, and tell me before continuing.
2. **Never delete or rerun a finished run to get a better number.** If a run must be redone for a technical reason (corrupted file, wrong GPU), move the old file to a folder `excluded`, write the reason in `DEVIATIONS.md`, and rerun.
3. **Do not lower a gate floor after seeing a failed gate.** A failed gate is a result to report.
4. **Keep the software the same.** Do not upgrade PyTorch, the NVIDIA driver or Python during the program. Every result file records the versions it ran with.
5. **One GPU type per backbone**, so the timing columns mean something.

8 When something goes wrong

what you see	what to do
PC rebooted, window closed, power cut	Run the same command again. It resumes.
`run failed (rc=1); stopping`	Open `train.log` in that backbone's folder and read the last 30 lines. Usually a full disk or a driver error. Fix it, rerun the same command. If unclear, send me the 30 lines.
`GATE FAILED <backbone>`	Nothing to do; the runner moved on. Note it. Do not lower the floor. Send me `GATE_FAILED.json` and the gate run's JSON; whether to try a tuned recipe is a separate, reported decision.
`DEVIATION ...: micro-batch 64 x 2`	Expected for big models on Tiny-ImageNet. Recorded and reported automatically.
`out of memory even at micro-batch 16`	Another program is using the GPU, or the GPU has less than 12 GB. Close other programs and rerun.
`DIVERGED` in the log	Expected occasionally. Do not rerun. It is counted and reported. More than three in one backbone: tell me.
`Tiny-ImageNet not prepared`	Run `python download_tinyimagenet.py -root .\data`, then the same command; it resumes there.
Disk full	Each result file is small; checkpoints are deleted when a run finishes. Free space, rerun. Keep at least 20 GB free.
`k left` has not moved for a day	Check `nvidia-smi`. If the GPU is idle, Ctrl+C and rerun the same command.

9 What to send me

After each backbone, or at the end:

```
Compress-Archive -Path .\results_models -DestinationPath results_models.zip
```

Send `results_models.zip` (result files, `runner.log`, `plan.json`, `DEVIATIONS.md`; it holds no checkpoints once runs finish) and the report PDFs. With those I can check every number and write up the cross-architecture section.

Where things are

file or folder	what
`run_all_models.py`	the runner (probe, gate, resume, divergence)
`src\program.py`	the 132-run program, defined once
`PREREGISTRATION_MODELS.md`	predictions, floors, rules (commit before running)
`README_MODELS.md`	the same commands as this guide, as text
`PARAMS_EXPECTED.md`	expected parameter count of every run
`COMPUTE_ESTIMATE.md`	the time estimate in Section 4
`results_models\<backbone>\`	results, `runner.log`, `train.log`, `plan.json`, `DEVIATIONS.md`
`report\out\`	the PDFs you build